

VULNERABILITY SCANNING AND REPORTING USING OWASP ZAP

Prepared By: Samuel Ochoche Peter

Date: June 2026

1. EXECUTIVE SUMMARY

This project involved conducting a vulnerability assessment against the Damn Vulnerable Web Application (DVWA) using the OWASP Zed Attack Proxy (ZAP) security testing tool. The purpose of the assessment was to identify common web application security weaknesses, evaluate their potential impact, and provide recommendations for remediation.

The vulnerability scan successfully identified several security issues including Application Error Disclosure, Missing Content Security Policy (CSP) Header, Missing Anti-Clickjacking Header, Information Leakage, and Cookie Security Weaknesses. These findings demonstrate the importance of secure coding practices and proper web server configuration in protecting applications against cyber threats.

The assessment was conducted within a controlled laboratory environment and followed industry best practices aligned with the OWASP Top 10 security framework.

2. PROJECT OBJECTIVES

The objectives of this assessment were to:

- Deploy and verify a vulnerable web application.
- Perform attack surface enumeration.
- Conduct automated vulnerability scanning using OWASP ZAP.
- Identify and categorize discovered vulnerabilities.
- Evaluate the risk associated with each finding.
- Recommend appropriate mitigation measures.
- Produce a professional vulnerability assessment report.

3. SCOPE OF ASSESSMENT

Target Application: Damn Vulnerable Web Application (DVWA)

Target URL: <http://localhost/dvwa>

Assessment Tool: OWASP ZAP Version 2.17.0

Environment: Windows 11 XAMPP Server Apache Web Server MySQL Database

Assessment Type: Automated Vulnerability Assessment

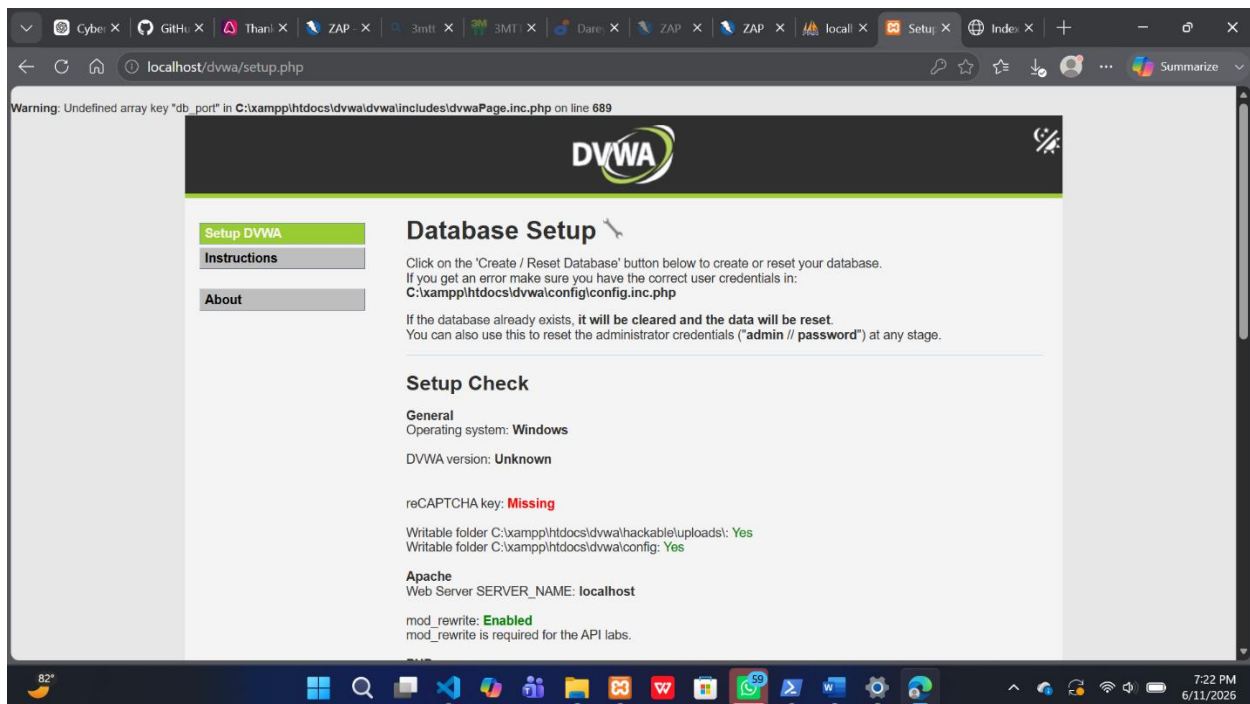
4. METHODOLOGY

The assessment followed the stages below:

4.1 Application Deployment

The DVWA application was installed and configured on a local XAMPP environment. Database connectivity was established and the application was verified to be operational.

Evidence: Figure 1 – Vulnerable Application Deployment

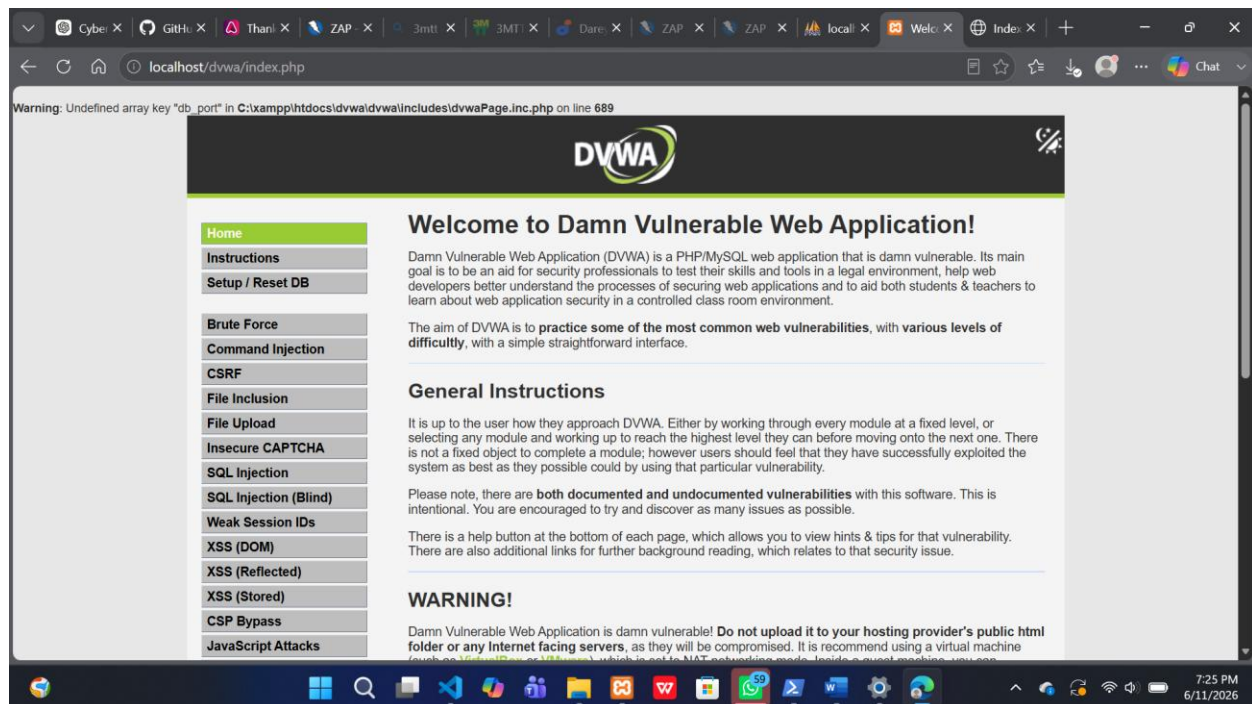


4.2 Attack Surface Enumeration

The application structure was examined to identify exposed modules and attack vectors. The following components were observed:

• Brute Force • Command Injection • CSRF • File Upload • SQL Injection • SQL Injection (Blind) • XSS (DOM) • XSS (Reflected) • XSS (Stored)

Evidence: Figure 2 – Application Attack Surface Enumeration

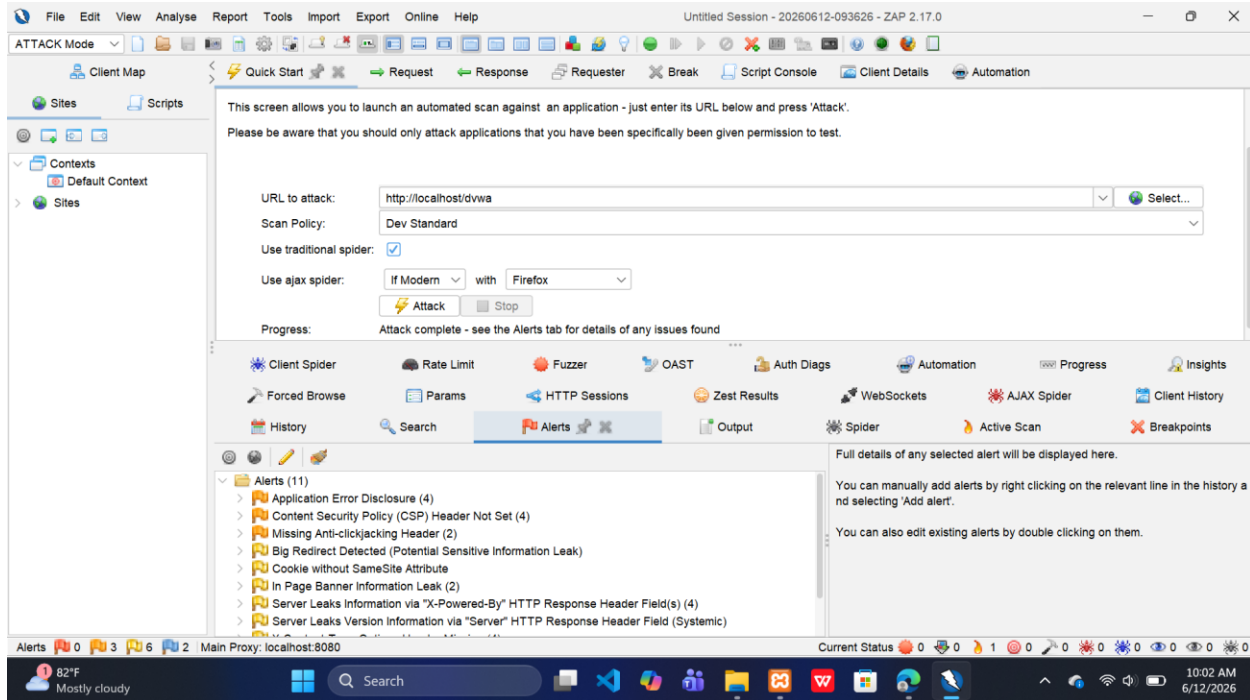


4.3 Automated Vulnerability Scanning

OWASP ZAP Automated Scan was executed against the application using the Dev Standard Scan Policy.

Target: <http://localhost/dvwa>

Evidence: Figure 3 – OWASP ZAP Automated Vulnerability Scan



4.4 Results Analysis

Discovered vulnerabilities were reviewed and categorized according to their risk level and potential business impact.

Evidence: Figure 4 – Vulnerability Alert Summary

5. VULNERABILITY FINDINGS

5.1 Application Error Disclosure

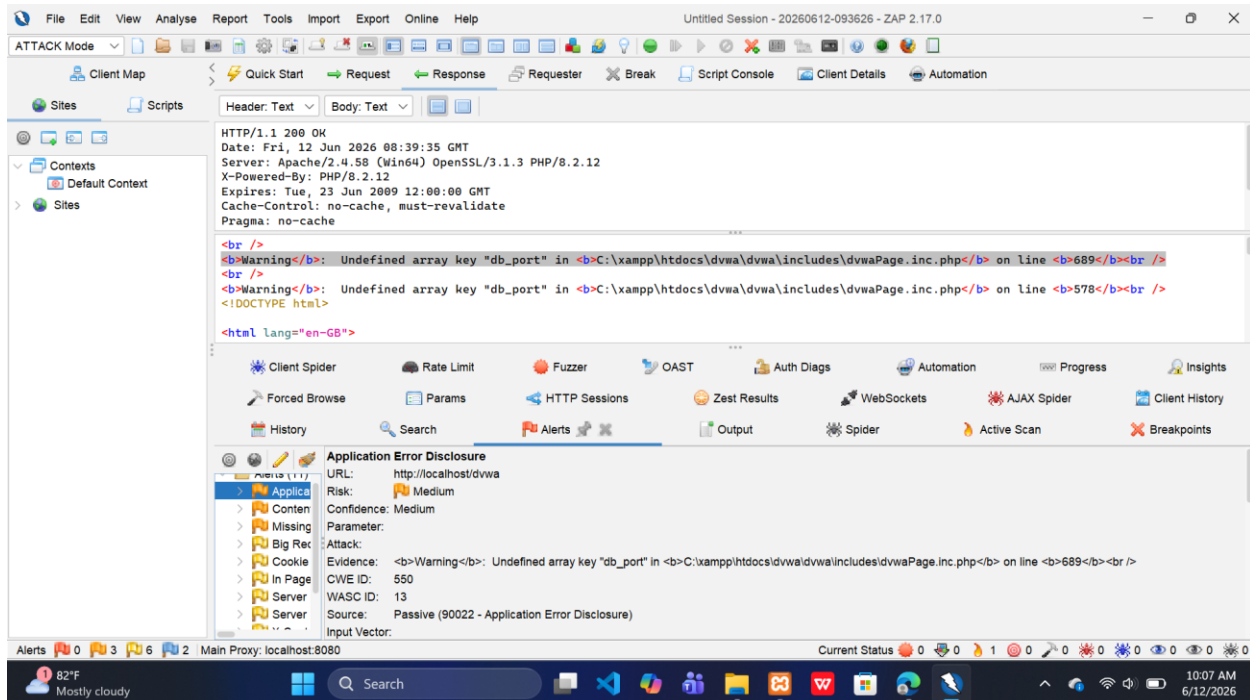
Risk Level: Medium

Description: The application exposes internal error messages to users. These messages may reveal sensitive information such as application structure, file paths, database details, or server configurations.

Potential Impact: • Information leakage • Reconnaissance assistance for attackers • Increased likelihood of targeted attacks

Recommendation: • Disable detailed error messages in production environments. • Implement centralized logging. • Display generic error messages to users.

Evidence: Figure 5 – Application Error Disclosure Vulnerability



5.2 Missing Content Security Policy (CSP) Header

Risk Level: Low

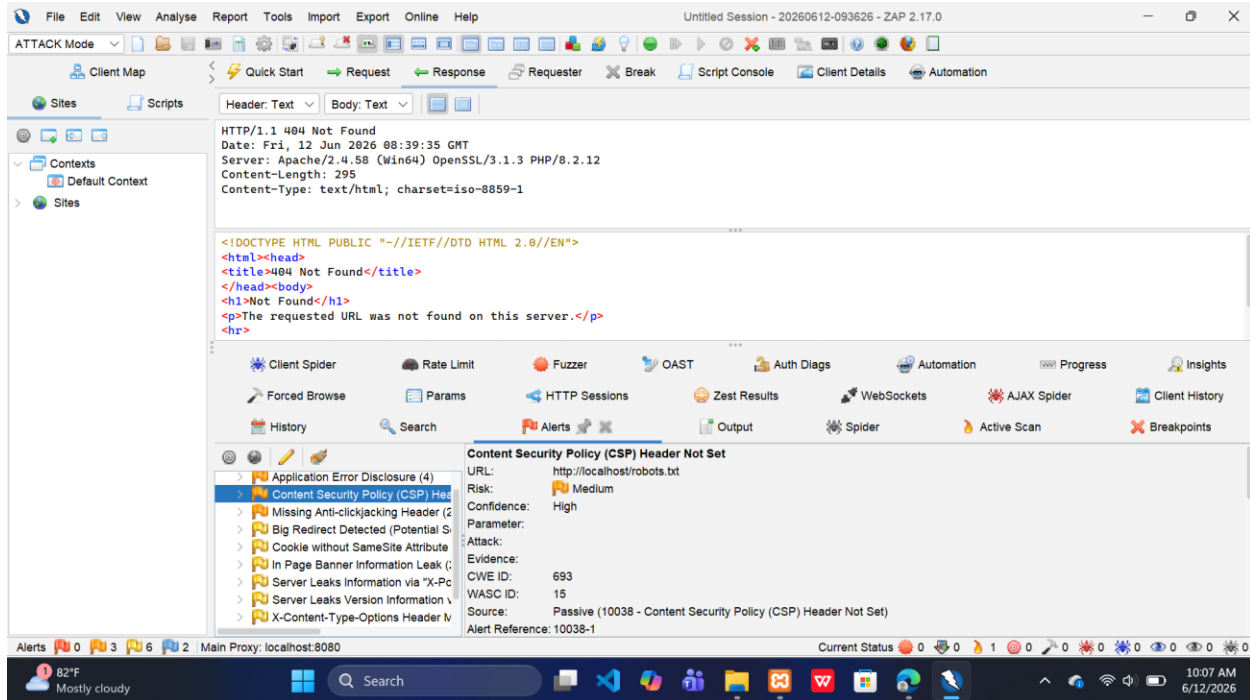
Description: The application does not implement a Content Security Policy header.

Potential Impact: • Increased susceptibility to Cross-Site Scripting (XSS) attacks. • Browser unable to restrict untrusted scripts.

Recommendation: Implement a Content Security Policy such as:

Content-Security-Policy: default-src 'self';

Evidence: Figure 6 – Missing Content Security Policy Header



5.3 Missing Anti-Clickjacking Header

Risk Level: Low

Description: The application does not use anti-clickjacking protections such as X-Frame-Options.

Potential Impact: • Clickjacking attacks. • Unauthorized user actions. • Session misuse.

Recommendation: Configure:

X-Frame-Options: DENY

or

Content-Security-Policy: frame-ancestors 'none';

Evidence: Figure 7 – Missing Anti-Clickjacking Header

The identified vulnerabilities primarily expose information that could assist attackers during reconnaissance activities. While no critical vulnerabilities were discovered during the scan, the combination of information disclosure and missing security headers increases the attack surface of the application.

Failure to address these issues may result in:

- Increased susceptibility to XSS attacks.
- Clickjacking attacks.
- Information leakage.
- Server fingerprinting.
- Session security weaknesses.

8. REMEDIATION PLAN

The following corrective actions are recommended:

1. Disable detailed error reporting in production systems.
2. Implement Content Security Policy (CSP) headers.
3. Configure Anti-Clickjacking protections.
4. Remove unnecessary server version disclosures.
5. Implement secure cookie attributes.
6. Regularly conduct vulnerability assessments.
7. Apply secure coding practices based on OWASP recommendations.
8. Maintain current security patches and updates.

9. CONCLUSION

This vulnerability assessment successfully demonstrated the process of identifying security weaknesses within a web application using OWASP ZAP. The scan revealed several medium and low-risk vulnerabilities associated with information disclosure and missing security controls.

The project reinforced practical skills in vulnerability assessment, risk analysis, remediation planning, and security reporting. Implementing the recommended security measures would significantly improve the application's security posture and reduce the likelihood of successful attacks.

REFERENCES

• OWASP Top 10 Web Application Security Risks • OWASP Zed Attack Proxy (ZAP) Documentation • NIST Cybersecurity Framework • OWASP Secure Coding Practices Guide • DVWA Documentation